

Reflection and Traceability Report on Software Engineering

Team 13, Speech Buddies
Mazen Youssef
Rawan Mahdi
Luna Aljammal
Kelvin Yu

This report provides a comprehensive reflection on the development and evolution of the VoiceBridge system from Revision 0 to Revision 1. The primary objective of this document is to demonstrate traceability between external feedback—received from teaching assistants, faculty advisors, and peer reviewers—and the subsequent refinements made to the system’s requirements, design, and verification protocols. This Document serves as a critical assessment of the engineering learning outcomes achieved throughout the Capstone project, including design iteration, project management efficiency, and the ethical considerations of developing assistive technologies.

1 Changes in Response to Feedback

1.1 SRS and Hazard Analysis

Over the course of the project, we made targeted updates to the SRS and Hazard Analysis in response to TA feedback, advisor guidance, and internal review. The main themes were: (i) improving requirement precision and testability, (ii) clarifying system boundaries and privacy assumptions for a local-first assistive tool, and (iii) strengthening safety language around command execution and user confirmation.

- **Clarified system boundary and privacy assumptions**

Source: TA/advisor feedback during Revision 0 review and demo discussions.

Feedback: The boundary between what runs locally vs. what relies on external components was not explicit enough, which made feasibility and privacy risk harder to assess.

Response: We revised the SRS to state the local-first intent more clearly, identify external dependencies where applicable, and tighten definitions

around what user data exists, where it can be stored, and what is not retained by default.

- **Improved requirement specificity and testability**

Source: TA feedback on requirement quality.

Feedback: Some requirements were too high-level or combined multiple behaviours, reducing testability and traceability.

Response: We reworded several requirements to be more measurable (clear triggers, observable outputs, and success criteria) and split bundled requirements so each one maps cleanly to design elements and V&V tests.

- **Strengthened traceability between SRS/HA and the design docs**

Source: TA feedback emphasizing traceability, including the design documentation review ([TA Review #235](#)).

Feedback: The design documentation needed clearer, more consistent module responsibilities and mappings, and the project artifacts needed to remain traceable across documents.

Response: While #235 primarily drove updates to the design documents, addressing it required us to cross-check the design against the SRS and Hazard Analysis. We verified and tightened SRS-to-module and hazard/mitigation-to-module traceability to reduce drift and ensure that safety and privacy assumptions in the SRS/HA were reflected in the architecture.

- **Expanded hazards related to unintended command execution**

Source: TA feedback and internal risk review after implementing command execution flows.

Feedback: The hazard analysis under-emphasized risks from misrecognition, ambiguous intent, and executing harmful browser actions.

Response: We strengthened hazards and mitigations around incorrect transcriptions and ambiguous commands leading to unintended actions. Mitigations were updated to emphasize explicit confirmation for higher-risk actions, clear user feedback during execution, and safe fallback behaviour when commands are out-of-scope or low-confidence.

- **Made privacy and data-retention hazards more concrete**

Source: TA/advisor feedback on privacy risk articulation.

Feedback: Privacy hazards were present but too abstract, making it unclear what data might be logged or retained and what controls exist.

Response: We clarified which artifacts (e.g., transcripts, recordings, logs, and user settings) might exist and refined mitigations to emphasize minimizing stored sensitive data, limiting log exposure, and treating stored artifacts as sensitive by default.

- **Addressed accessibility-related failure modes as safety/usability risks**

Source: TA feedback and usability-oriented review.

Feedback: Since the target users include individuals with dysarthric speech, the analysis should consider failure modes such as unclear prompts, missing confirmations, or confusing system state.

Response: We added/refined hazards capturing user confusion and loss of control due to insufficient feedback, and we connected mitigations to clearer state indications and consistent confirm/cancel interactions.

1.2 Design and Design Documentation

Most of our feedback-driven changes were in the design documentation, where we focused on improving clarity, consistency, and alignment with the implemented system.

- **Improved module boundaries and responsibility descriptions**

Source: TA feedback on ambiguity/overlap between components.

Feedback: Some responsibilities were described in overlapping terms (e.g., session flow vs. command interpretation vs. browser execution), making it hard to understand component ownership.

Response: We revised the Module Guide’s module descriptions to better separate responsibilities (UI/feedback vs. transcription vs. command synthesis vs. execution/orchestration vs. storage/security). This made the architecture easier to understand and reduced ambiguity for implementation and testing.

- **Updated documentation to match the final implementation**

Source: TA/advisor feedback plus internal consistency checks.

Feedback: Some design text reflected earlier architecture drafts rather than the final code structure, creating “documentation drift.”

Response: We updated module descriptions and terminology so they accurately reflect the final architecture and how the system is implemented (including details of the audio capture flow and the model tuning workflow). This improved the MG’s usefulness as a maintenance reference.

- **Strengthened traceability matrices and requirement coverage**

Source: TA feedback emphasizing traceability and completeness.

Feedback: The design needed clearer evidence that requirements (functional, usability, performance, security, etc.) were mapped to concrete parts of the system.

Response: We revised and validated the traceability matrices so each requirement category maps to the relevant modules, and we checked for

gaps/duplicates. This also helped ensure the V&V work could be planned directly against the design.

- **Clarified inter-module communication and error handling**

Source: Advisor/TA feedback requesting clearer interaction details.

Feedback: The documents needed clearer description of how modules exchange information and how errors propagate to user-facing feedback.

Response: We expanded the design documentation to describe the message/data flow at a high level (what data types are passed, where session state is tracked, and how failures are turned into user-facing feedback and logs). This made the design more actionable and easier to verify.

- **Refined UI and accessibility documentation based on feedback**

Source: TA feedback and informal user-focused review.

Feedback: The UI story needed clearer user-state feedback (listening, processing, confirm/cancel) and accessibility justification.

Response: We improved UI-related sections and figures/descriptions to emphasize predictable state transitions, clear confirmations, and accessible feedback. These changes were motivated by the needs of users with dysarthric speech, where system confidence and clarity directly affect usability.

1.3 VnV Plan and Report

The VnV Plan and associated report were revised to incorporate feedback from peer reviews and formal TA review. The changes focused on improving traceability, making nonfunctional tests more specific, clarifying the scope of extras, and refining document language for readability.

Table 1: VnV Report

Issue	Response and Change
Peer Review #91	Expanded the VnV Plan traceability matrix so every nonfunctional test case is explicitly linked to the corresponding SRS nonfunctional requirement. This made the relationship between look-and-feel tests, usability/humanity tests, and the SRS clear.
Peer Review #92	Proofread and corrected grammar and structure throughout the VnV documents. We simplified long sentences, improved section flow, and made the VnV Plan easier to follow.
Peer Review #93	Added concrete test details for nonfunctional cases. In particular, the ASR latency benchmark now includes the source and selection method for audio samples, the measurement procedure, and the expected results.
Peer Review #94	Clarified extras feasibility and marked as wont fix per TA/instructor instructions.
Peer Review #95	Added detailed nondynamic testing plans, including roles for code inspections, document reviews, and static analysis in Sections 3.6 and 3.7.
Peer Review #96	Expanded Section 2.2 with testable objectives linked to critical qualities. Revised Section 2.3 to identify extra verification activities and their feasibility.
TA Review #116	Addressed TA comments on the VnV Plan and report by refining the document structure, improving traceability, and tightening wording to better align with the review feedback.

The VnV Report was revised to incorporate feedback from peer reviews. The changes focused on improving traceability and providing explanations for test results.

Table 2: VnV Report

Issue	Response and Change
Peer Review #286	Clarified changes made due to testing by directly linking them to specific test results and outcomes.
Peer Review #287	Updated test results to show actual quantitative data and align with VnV Plan pass criteria.
Peer Review #288	Added missing test cases (test-LF-1, test-LF-2, test-OER-1, test-OER-2) to the VnV Report.
Peer Review #289	Added test FR6 to the VnV Plan to maintain traceability between the plan and report documents.
Peer Review #290	Explained the deferral of test-PF-3 as a stretch goal and updated the VnV Plan accordingly.
Peer Review #291	Provided explanation for the failing result of test-PF-1, including next steps to resolve the issue and clarify the non-functional requirement.
TA Review #320	Addressed TA comments on the VnV Report by refining structure, results presentation, and alignment with the VnV Plan.

2 Extras

Our team completed the two following extras:

Document	Methodology	Key Focus Area
Literature Review	Semi-systematic synthesis of recent ASR research, technical reports, and working notes.	Evaluation of fine-tuning strategies, parameter-efficient methods (LoRA), and specialized preprocessing for dysarthric speech.
ML Report	Technical implementation and parameter-efficient fine-tuning (PEFT) using Whisper-small.	On-device adaptation, local privacy-preserving workflows, and mitigating catastrophic forgetting via replay buffers.

Table 3: Summary of VoiceBridge Research and Technical Documentation

3 Design Iteration (LO11 (PrototypeIterate))

3.1 Audio Capture Modules

A key iteration in our system design was consolidating the modules related to audio capture, including the former MicrophoneManager, VADCapture, etc. into one master module. The reason for this is that our team came across pre-existing open source solutions in the form of python packages that could simplify our design. The packages were the following:

Library	Primary Function	Key Features
<code>sounddevice</code>	Real-time audio I/O and hardware interfacing.	Provides bindings for the PortAudio library; handles multi-channel recording, low-latency streaming, and device selection.
<code>webrtcvad</code>	Voice Activity Detection (VAD).	Specifically designed to distinguish between human speech and background noise; uses the high-performance WebRTC VAD engine.

Table 4: Core Audio Processing Libraries for VoiceBridge

Given these two libraries covered microphone/audio device management as well as VAD filtering, we were able to depend on them entirely rather than

creating our own modules. Thus, we were able to create a single consolidated class for collecting audio data, simplifying our design and process.

3.2 Shortcut Manager Iteration

One important design iteration in VoiceBridge was the development of the **ShortcutManager**. In the earlier versions of the system, commands were handled one at a time with no persistent way to record, save, and replay common action sequences. While this was enough to demonstrate basic browser control, it did not match the client need for efficiency and accessibility in repeated everyday tasks. For a user with impaired speech, repeatedly issuing the same sequence of voice commands can increase fatigue and reduce the overall usability of the system. Because of this, we introduced shortcuts as a way to reduce the number of spoken interactions required for frequent tasks.

Our first idea was relatively simple: store a shortcut as a sequence of recorded commands and replay them later. However, once we integrated this into the live system, we found that shortcut behavior could conflict with the runtime state machine. For example, the user could try to begin a shortcut recording while the system was still processing another command, or multiple parts of the backend could attempt to modify shortcut state at the same time. This was especially important because VoiceBridge is a real-time system with asynchronous events, including speech capture, transcription, intent resolution, and browser execution. As a result, we revised the design so that shortcut recording became an explicitly managed subsystem rather than just an added list of commands.

The final design introduced a dedicated **ShortcutManager** module with a clear internal state, persistent storage, and locking for safe concurrent access. Instead of scattering shortcut logic across the codebase, the manager became responsible for starting and stopping recordings, storing recorded commands, validating shortcut names, and loading/saving shortcuts from disk. This design was chosen to improve maintainability and prevent race conditions in cases where multiple backend events happen close together. The use of a lock was an important iteration because testing showed that shortcut actions could overlap with other command-processing flows in unpredictable ways if the state was not protected.

We also changed how shortcuts were represented. In earlier thinking, shortcuts were treated mainly as temporary in-memory recordings for the active session. This was limiting, because a user would lose their shortcuts between runs of the application. Since one of the client goals was to make the system practical for daily use, we moved to persistent storage using a local JSON-based approach. This allowed shortcuts to survive application restarts and made the feature more realistic as an assistive tool rather than just a demo feature. The trade-off was that we now needed validation and structured file handling, but this was worth it because it supported a more reliable user experience.

Another design change was motivated by usability. Initially, shortcut support was focused only on storing the raw sequence of actions. As the design matured, we recognized that the feature also needed clear start/stop control

and user feedback so that recording would feel predictable. Without this, a user could be unsure whether the system was actively recording commands into a shortcut or just executing them normally. This was especially important given our client focus on accessibility and confidence in system behavior. We therefore refined the design so that shortcut recording had explicit transitions, clear prompts, and tighter integration with the system state flow.

Overall, the **ShortcutManager** evolved from a simple convenience idea into a more structured subsystem designed around accessibility, predictability, and safe execution. The main reason for these changes was not just technical cleanup, but the need to better support users who benefit from reducing repeated speech effort while still being able to trust what the system is doing.

4 Design Decisions (LO12)

4.1 Iterative Model Improvements

A key feature of VoiceBridge was developed after our Rev0 version, that being the gradual model customization and improvements through on device training. This feature introduced a set of novel challenges for our project.

4.1.1 Privacy-Preserving Local Adaptation

The primary challenge was leveraging user-specific data while maintaining strict privacy standards. To resolve this, we adopted an **on-device processing architecture**. No raw audio recordings or transcripts are transmitted to external servers; instead, all fine-tuning and inference tasks are executed locally on the user’s hardware.

4.1.2 Computational Efficiency on Consumer Hardware

While the chosen base model is relatively compact, executing machine learning training remains resource-intensive for standard consumer-grade CPUs. We addressed this through a three-fold optimization strategy:

- **Low-Rank Adaptation (LoRA):** As previously detailed, LoRA is utilized to drastically reduce the number of trainable parameters.
- **Targeted Decoupling:** Training is restricted exclusively to the **decoder** portion of the Whisper architecture—where language and vocabulary adaptation occur—while the **encoder** remains frozen. This focuses the computational effort on user-specific linguistic patterns while preserving general acoustic features.
- **Incremental Batching:** Rather than retraining on the entire accumulated dataset, the system performs small, incremental updates. Training is triggered after collecting several hundred new samples and is designed to complete within a short window (typically under 20 minutes), allowing for gradual improvement without intensive resource sessions.

4.1.3 System Responsiveness and Latency

Executing local retraining on consumer hardware poses a risk of system instability or "bogging down" the primary interface. To maintain a fluid user experience, the application isolates the training pipeline within a `**background thread**`. This asynchronous execution ensures that the main application logic and user interface remain responsive and uninterrupted during the adaptation process.

4.1.4 Mitigating Catastrophic Forgetting

A final challenge involves maintaining the model's general accuracy during iterative updates. In machine learning, repeated training on limited new data can lead to **Catastrophic Forgetting**, where the model loses previously acquired patterns. To mitigate this, we implemented a **replay buffer system**. A subset of historically relevant samples is retained and interleaved with new data during each training cycle. This ensures the model effectively incorporates new user-specific patterns while preserving its foundational knowledge and overall transcription stability.

5 Economic Considerations (LO23)

Given our product is open source, there are limited economic considerations. Marketing our product would involve community outreach and promotion on public forums that can reach individuals with dysarthria. This may include getting it touch with organizations that are specialized in speech accessibility, or simply online forums like reddit. These initiatives would be done with the goal of recruiting participants and open source developers to further the research and development of the product rather than to sell it or generate profit.

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

Our project management aligned well with our Development Plan. We maintained a consistent meeting rhythm to coordinate priorities and remove blockers, and we used asynchronous communication between meetings to keep work moving. We used GitHub Issues and pull requests to plan tasks, record feedback, and track progress across deliverables. Team roles were clear and helped us work efficiently, while still allowing collaboration across areas when needed. We also used the technologies we planned on using (GitHub for version control and coordination, and LaTeX for documentation), and we kept our workflow structured around reviewable changes and traceability.

6.2 What Went Well?

- **Issue tracking and traceability:** Capturing peer and TA feedback as GitHub Issues made it easy to plan changes and clearly demonstrate traceability from comments to updates.
- **Clear ownership with collaboration:** Assigning primary owners for major deliverables improved consistency and helped each document progress steadily, while collaboration ensured quality and shared understanding.
- **Review and iteration:** Regular internal review cycles improved clarity, consistency, and alignment across the SRS/HA, design documentation, and V&V artifacts.
- **Coordination across implementation and documentation:** Keeping documentation connected to the implementation helped ensure design decisions were well-justified and supported by the final system.

6.3 What Went Wrong?

- **Continuous alignment effort:** Maintaining alignment between requirements, design, and V&V artifacts required ongoing attention, and the experience reinforced the value of regularly revisiting traceability links as the system evolves.
- **Nonfunctional evaluation planning:** Defining measurable nonfunctional tests and presenting quantitative results clearly was a valuable learning area that strengthened the quality of our V&V work.
- **Estimating integration work:** Coordinating integration across components highlighted the importance of explicitly planning interface details and test hooks early, which improved the final system robustness.

6.4 What Would you Do Differently Next Time?

- **Make traceability updates routine:** Update traceability matrices on a regular schedule so cross-document consistency is maintained continuously.
- **Define acceptance criteria for issues:** Add short, explicit acceptance criteria to issues to keep scope clear and improve verification.
- **Plan earlier nonfunctional baselines:** Establish performance and usability baselines earlier so evaluation is repeatable and comparisons across iterations are straightforward.

7 Ethics and Equity (LO18)

Equity and universal design were central to VoiceBridge because the system is intended to support individuals with dysarthric speech. We designed for clarity, predictability, and user control by emphasizing clear system-state feedback (listening, processing, executing), confirmation for higher-risk actions, and safe handling of low-confidence or out-of-scope commands. We also treated privacy as part of equitable design by documenting data assumptions and encouraging local-first handling where feasible, so users can understand and trust how their information is processed. Across the project, we aimed to reduce unnecessary cognitive load and promote confidence by keeping interactions consistent and transparent.

8 Reflection on Capstone

8.1 Which Courses Were Relevant

- **SFWRENG 4HC3 (Human-Computer Interaction):** Informed our focus on usable, predictable interactions, user feedback, and accessibility-oriented design decisions.
- **SFWRENG 3A04 (Software Architecture):** Supported our approach to module decomposition, clear responsibility assignment, and documenting interfaces and system structure.
- **SFWRENG 4AL3 (Applied Machine Learning):** Helped us reason about ASR-related considerations such as evaluation, error modes, and performance tradeoffs in practical settings.

8.2 Knowledge/Skills Outside of Courses

- **End-to-end ASR pipeline integration:** Building and integrating a real-time flow from audio capture to transcription to command execution.
- **Practical evaluation:** Designing repeatable procedures and metrics (e.g., latency and error characterization) that reflect real usage.
- **Privacy-aware engineering:** Translating privacy expectations into concrete requirements, hazards, and mitigations for an assistive tool.
- **Documentation and maintenance discipline:** Keeping requirements, design, and V&V artifacts consistent through traceability as the project evolved.
- **Collaborative workflow:** Applying issue tracking, pull requests, and structured reviews to coordinate work and justify changes.